

Frontend studyloop

Sunday, September 27, 2026 3:48 PM

📖 StudyLoop — الملخص الكامل

1. ما هي فكرة المشروع؟

StudyLoop هو Web App لإدارة التعلّم والمراجعة

المشكلة الأساسية التي يحلّها

الطالب يدرس أشياء كثيرة، لكنه بعد فترة ينسى ما درسه ولا يعرف ماذا يراجع ومعى

يحاول يعمل دورة كاملة StudyLoop، Notes App، فينبدل ما يكون التطبيق مجرد

Study → Organize → Review → Test → Track Progress → Ask AI

يعني المستخدم لا يخزّن المعلومات فقط، بل يتابع عملية تعلّمه واحتفاظه بالمعلومات

المشكلة التي يحلها المشروع 2.

المشكلة فيها عدة أجزاء

المعلومات تضيع — المشكلة 1

الطالب يدرس

DI
SOLID
REST
PostgreSQL
System Design

وبعد أسبوعين يريد أن يعرف

"شو كنت دارس؟"

ينظم هذه المعرفة داخل StudyLoop

Container → Lesson → Topics

لا يعرف ماذا أنجز — المشكلة 2

كاملة، لكن لا يعرف Lesson قد يقرأ

فهمها؟ Topics أي
لم يراجعها؟ Topics أي
كم نسبة إتجازة؟

لذلك لدينا

UserTopicProgress + Progress

نفسه Topic بحيث حالة التعلّم مرتبطة بالمستخدم، وليس بالـ

ينسى ما درسه — المشكلة 3

وهذه أهم مشكلة

، لا يعني أن المستخدم احتفظ بالمعلومة Lesson بعد إنهاء

لذلك يوجد

Review / Spaced Repetition

ويتم تحديد موعد المراجعة التالية

بسيطة intervals حالياً الـ

1 → 3 → 7 → 14 → 30 → 60 days

أحياناً لا يعرف ماذا يسأل — المشكلة 4

المستخدم أثناء الدراسة قد يقول

"What is Dependency Injection?"

أو:

"Why do we use DIP?"

لذلك يوجد

Ask AI

ويستطيع السؤال على مستوى

Lesson
Topic
الدراسة ليست فقط قراءة — المشكلة 5

المستخدم يحتاج أن يختبر نفسه

لذلك لدينا

AI-generated Quizzes

Quiz ويمكن إنشاء

كاملة Lesson للـ

محددة Topics أو لمجموعة

مع:

questions
options
answers
score
يريد معرفة تطوره — المشكلة 6

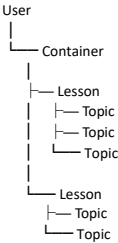
لذلك لدينا:

Analytics

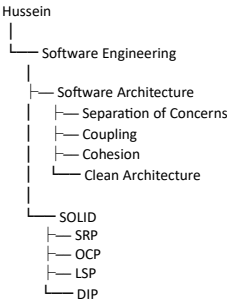
تعرض معلومات مثل

Lessons عدد ال
Topics عدد ال
المكتملة Topics
Global completion rate
Quizzes عدد ال
Average score
Recent activity
Annotations
3. ال Core Structure

الأساسية hierarchy ال



مثلاً:



4. أهم ال Features
Authentication

المستخدم يستطيع:

Register
Login
Get current user

مع:

JWT
Password hashing
Protected routes
Validation
Containers

Lessons. هو طريقة لتنظيم ال Container ال

مثلاً:

Software Engineering
Web Development
University
LeetCode

Lessons. يحتوي عدة Container وال

Features:

Create
Read
Update
Delete
Lessons

.هي الوحدة التعليمية الأساسية Lesson ال

مثلاً:

Lesson: Dependency Injection

- Topics:
- What is DI?
 - Why DI?
 - Dependency Inversion
 - DI in NestJS

Features:

Create
Read
Update
Delete
Add topics
Update topics
Delete topics
Share lesson publicly
بطريقتين Lesson إنشاء

وهذه من الأفكار المهمة بالمشروع

Structured — الطريقة الأولى

مباشرة Topics المستخدم يدخل:

Title: Dependency Injection

- Topics:
1. What is DI?
 2. Why DI?
 3. Dependency Inversion
- Raw Content — الطريقة الثانية

نص طويل paste المستخدم يستطيع

Dependency Injection is a design pattern...

It helps us...

There are several reasons...

يقوم بتحليل النص وتحويله إلى AI وال

Lesson
├── Topic
├── Topic
├── Topic
└── Topic

CRUD. وهذا يجعل التطبيق أكثر من مجرد

AI Features

جزء مهم من المشروع AI ال

1. Parse Raw Content

تحويل:

Unstructured text → Structured Lesson

2. Ask AI

Topic أو Lesson المستخدم يسأل عن

مثلاً:

Why is dependency injection useful?

يجيب بناءً على سياق الدرس AI وال

3. Generate Quiz

يولّد أسئلة بناءً على المحتوى AI ال

مثلاً:

Lesson: SOLID

Question:

What does SRP stand for?

- A. Single Responsibility Principle
B. Software Responsibility Pattern
C. ...

ويمكن تحديد:

Scope
Difficulty
Format
Number of questions
Progress System

architecture. هذه من أهم أجزاء ال

عندنا فصل واضح بين

Topic

يمثل المحتوى نفسه

Topic

لا يحتوي على:

isCompleted

واحد يمكن أن يدرسه آلاف المستخدمين Topic لأن

UserTopicProgress

يمثل:

Topic حالة هذا المستخدم تجاه هذا الـ

مثلاً:

User A → Topic 1 → completed

User B → Topic 1 → not completed

وفيه:

isCompleted

completedAt

Progress

Lesson يمثل التقدم العام للمستخدم داخل

ويحتوي على:

completedTopicsCount

totalTopicsCount

completionPercentage

scrollPosition

isLessonCompleted

lastReviewedAt

nextReviewDate

reviewCount

reviewIntervalDays

وبالتالي:

Topic

↓

UserTopicProgress

↓

Progress

 Review System

Lesson بعد التعلم، المستخدم يستطيع مراجعة

النظام يحتفظ بـ

lastReviewedAt

nextReviewDate

reviewCount

reviewIntervalDays

حالياً review intervals والـ

1 day

3 days

7 days

14 days

30 days

60 days

ويوجد:

Due Reviews

التي حان وقت مراجعتها Lessons بعيد الـ Endpoint

يعني المستخدم يستطيع فتح

Reviews Due Today

ويعرف ماذا يجب أن يراجع

 Analytics

Dashboard Analytics لدينا

يعطي:

Total Lessons

Total Topics

Completed Topics

Global Completion Rate

Total Annotations

وأيضاً:

Recent Activity

والتقدم فيها Lessons يعرض آخر

مثلاً:

Clean Architecture

7 / 10 topics

70%
SOLID
5 / 5 topics
100%
Quiz Statistics

مثل:

Total Quizzes Taken
Average Score
Annotations

Lesson على Annotation المستخدم يستطيع إضافة

مثلاً:

"Important:
DIP does not mean Dependency Injection."

Topic يمكن أن يرتبط ب Annotation وال

Features:

Create
Read
Update
Delete

User B الخاصة ب Annotation لا يستطيع تعديل User A مرتبطة بالمستخدم، لذلك annotations وال

Quizzes

يحتوي على Quiz ال

Quiz
|— Question
|— Question
|— Question

فيها Questions وال

questionText
format
order
options

يعرف Quiz وال

Lesson
Scope
Difficulty
Format
Question count

Quiz statistics وفيه أيضاً

Ask Questions

لدينا أيضاً نظام لحفظ أسئلة المستخدم

Question
↓
Answer

المستخدم يستطيع

Ask
View questions
View a specific question
Delete question

يتم حفظه AI answer وال

Public Sharing

Lesson المستخدم يستطيع جعل

Public

ثم يحصل على

shareToken

بدون تسجيل دخول Lesson ويصبح بالإمكان مشاركة

مثلاً:

/lessons/share/abc123

JWT لا يحتاج Public endpoint وال

Security / Ownership

من أهم الأشياء التي عملنا عليها

ID. المستخدم لا يستطيع الوصول إلى بيانات مستخدم آخر فقط لأنه يعرف ال

مثلاً:

User A

Lesson A

لا يستطيع

PATCH Lesson B
DELETE Annotation B
GET Progress B
Create Quiz for Lesson B

تتحقق من Services لأن الـ

userId
↓
container ownership
↓
lesson ownership / relation
↓
topic relation

domain boundaries والـ authorization فقط، بل تفكر بالـ CRUD وهذا جزء مهم جداً من المشروع لأنه يبين أنك لا تبني

AuthModule:

```
import {
  Controller,
  Post,
  Body,
  Get,
  UseGuards,
  Request as NestRequest,
} from '@nestjs/common';
import type { Request } from 'express';
import { AuthService } from './auth.service.js';
import { RegisterDto } from './dto/register.dto.js';
import { LoginDto } from './dto/login.dto.js';
import { JwtGuard } from './guards/jwt.guard.js';
interface AuthenticatedRequest extends Request {
  user: {
    sub: string;
  };
}
@Controller('auth')
export class AuthController {
  constructor(private readonly authService: AuthService) {}
  /**
   * Register a new user
   * POST /auth/register
   */
  @Post('register')
  async register(@Body() registerDto: RegisterDto) {
    return this.authService.register(registerDto);
  }
  /**
   * Login user
   * POST /auth/login
   */
  @Post('login')
  async login(@Body() loginDto: LoginDto) {
    return this.authService.login(loginDto);
  }
  /**
   * Get current user profile
   * GET /auth/me
   */
}
```

```

    * Protected with JWT
    */
    @Get('me')
    @UseGuards(JwtGuard)
    async getCurrentUser(@NestRequest() req: AuthenticatedRequest) {
        return this.authService.getCurrentUser(req.user.sub);
    }
}

```

Login dto :

```

import { IsEmail, IsNotEmpty, IsString, MinLength } from 'class-validator';
export class LoginDto {
    @IsEmail({}, { message: 'Please provide a valid email address' })
    @IsNotEmpty({ message: 'Email is required' })
    email: string;
    @IsString()
    @IsNotEmpty({ message: 'Password is required' })
    @MinLength(8, { message: 'Password must be at least 8 characters long' })
    password: string;
}

```

Register dto :

```

import {
    IsEmail,
    IsNotEmpty,
    IsString,
    MinLength,
    Matches,
} from 'class-validator';
export class RegisterDto {
    @IsEmail({}, { message: 'Please provide a valid email address' })
    @IsNotEmpty({ message: 'Email is required' })
    email: string;
    @IsString()
    @IsNotEmpty({ message: 'Password is required' })
    @MinLength(8, { message: 'Password must be at least 8 characters long' })
    @Matches(
        /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]+$/,
        // <-- Zidna +$ bil-2aakher
    )
    message:
        'Password must contain uppercase, lowercase, number, and special character',
    },
    ),
    password: string;
    @IsString()
    @IsNotEmpty()
    firstName: string;
    @IsString()
    @IsNotEmpty()
    lastName: string;
}

```

Container module:

```

import {
    Controller,
    Post,
    Get,
    Put,
    Delete,
    Body,
    Param,
    UseGuards,
    Request as NestRequest,
} from '@nestjs/common';
import type { Request } from 'express';
import { ContainerService } from '../container.service.js';
import { CreateContainerDto } from '../dto/create-container.dto.js';
import { UpdateContainerDto } from '../dto/update-container.dto.js';
import { JwtGuard } from '../auth/guards/jwt.guard.js';
interface AuthenticatedRequest extends Request {

```

```

    user: {
      sub: string;
    };
  }
}
@Controller('containers')
@UseGuards(JwtGuard)
export class ContainerController {
  constructor(private readonly containerService:
ContainerService) {}
  // Create a new container -> POST /containers
  @Post()
  async createContainer(
    @Body() createContainerDto: CreateContainerDto,
    @NestRequest() req: AuthenticatedRequest,
  ) {
    return this.containerService.createContainer(
      req.user.sub,
      createContainerDto,
    );
  }
  /**
   * Get all containers for the current user
   * GET /containers
   */
  @Get()
  async getContainers(@NestRequest() req: AuthenticatedRequest)
  {
    return this.containerService.getContainers(req.user.sub);
  }
  /**
   * Get a specific container
   * GET /containers/:id
   */
  @Get('/:id')
  async getContainer(
    @Param('id') containerId: string,
    @NestRequest() req: AuthenticatedRequest,
  ) {
    return this.containerService.getContainer(containerId,
req.user.sub);
  }
  /**
   * Update a container
   * PUT /containers/:id
   */
  @Put('/:id')
  async updateContainer(
    @Param('id') containerId: string,
    @Body() updateContainerDto: UpdateContainerDto,
    @NestRequest() req: AuthenticatedRequest,
  ) {
    return this.containerService.updateContainer(
      containerId,
      req.user.sub,
      updateContainerDto,
    );
  }
  /**
   * Delete a container
   * DELETE /containers/:id
   */
  @Delete('/:id')
  async deleteContainer(
    @Param('id') containerId: string,
    @NestRequest() req: AuthenticatedRequest,
  ) {
    return this.containerService.deleteContainer(containerId,
req.user.sub);
  }
}

```

Create container:

```

import { IsNotEmpty, IsString, IsOptional } from 'class-
validator';
export class CreateContainerDto {
  @IsString()
  @IsNotEmpty({ message: 'Container name is required' })
  name: string;
  @IsString()
  @IsOptional()
  description?: string;
}

```

Update container :

```
import { IsNotEmpty, IsString, IsOptional } from 'class-validator';
export class UpdateContainerDto {
  @IsString()
  @IsOptional()
  @IsNotEmpty()
  name?: string;
  @IsString()
  @IsOptional()
  description?: string;
}
```

Lesson Module
Cnontroller:

```
import {
  Controller,
  Post,
  Get,
  Put,
  Patch,
  Delete,
  Body,
  Param,
  UseGuards,
  Request as NestRequest,
} from '@nestjs/common';
import type { Request } from 'express';
import { LessonService } from '../lesson.service.js';
import { CreateLessonDto } from '../dto/create-lesson.dto.js';
import { UpdateLessonDto } from '../dto/update-lesson.dto.js';
import { UpdateTopicDto } from '../dto/update-topic.dto.js';
import { JwtGuard } from '../auth/guards/jwt.guard.js';
import { UpdateLessonShareDto } from '../dto/update-lesson-share.dto.js';
import { ParseRawTextDto } from '../dto/parse-raw-text.dto.js';
interface AuthenticatedRequest extends Request {
  user: {
    sub: string;
  };
}
@Controller('containers/:containerId/lessons')
@UseGuards(JwtGuard)
export class LessonController {
  constructor(private readonly lessonService: LessonService) {}
  /**
   * Create a new lesson with topics
   * POST /containers/:containerId/lessons
   */
  @Post()
  async createLesson(
    @Param('containerId') containerId: string,
    @Body() createLessonDto: CreateLessonDto,
    @NestRequest() req: AuthenticatedRequest,
  ) {
    return this.lessonService.createLesson(
      containerId,
      req.user.sub,
      createLessonDto,
    );
  }
  /**
   * Get all lessons in a container
   * GET /containers/:containerId/lessons
   */
  @Get()
  async getLessonsByContainer(
    @Param('containerId') containerId: string,
    @NestRequest() req: AuthenticatedRequest,
  ) {
    return
    this.lessonService.getLessonsByContainer(containerId,
    req.user.sub);
  }
  /**
   * Get a specific lesson
   * GET /containers/:containerId/lessons/:id
   */
  @Get('/:id')
  async getLesson(
    @Param('containerId') containerId: string,
    @Param('id') lessonId: string,
```

```

    @NestRequest() req: AuthenticatedRequest,
  ) {
    return this.lessonService.getLesson(lessonId, containerId,
req.user.sub);
  }
}
/**
 * PUT /containers/:containerId/lessons/:id
 */
@Put('/:id')
async updateLesson(
  @Param('containerId') containerId: string,
  @Param('id') lessonId: string,
  @Body() updateLessonDto: UpdateLessonDto,
  @NestRequest() req: AuthenticatedRequest,
) {
  return this.lessonService.updateLesson(
    lessonId,
    containerId,
    req.user.sub,
    updateLessonDto,
  );
}
/**
 * Delete a lesson
 * DELETE /containers/:containerId/lessons/:id
 */
@Delete('/:id')
async deleteLesson(
  @Param('containerId') containerId: string,
  @Param('id') lessonId: string,
  @NestRequest() req: AuthenticatedRequest,
) {
  return this.lessonService.deleteLesson(lessonId,
containerId, req.user.sub);
}
/**
 * Update a topic (title, description, completion status)
 * PUT
 /containers/:containerId/lessons/:lessonId/topics/:topicId
 */
@Put('/:lessonId/topics/:topicId')
async updateTopic(
  @Param('containerId') containerId: string,
  @Param('lessonId') lessonId: string,
  @Param('topicId') topicId: string,
  @Body() updateTopicDto: UpdateTopicDto,
  @NestRequest() req: AuthenticatedRequest,
) {
  return this.lessonService.updateTopic(
    topicId,
    lessonId,
    containerId,
    req.user.sub,
    updateTopicDto,
  );
}
/**
 * Delete a topic
 * DELETE
 /containers/:containerId/lessons/:lessonId/topics/:topicId
 */
@Delete('/:lessonId/topics/:topicId')
async deleteTopic(
  @Param('containerId') containerId: string,
  @Param('lessonId') lessonId: string,
  @Param('topicId') topicId: string,
  @NestRequest() req: AuthenticatedRequest,
) {
  return this.lessonService.deleteTopic(
    topicId,
    lessonId,
    containerId,
    req.user.sub,
  );
}
}
@Post('parse-raw')
async parseRawText(@Body() dto: ParseRawTextDto) {
  return this.lessonService.parseRawText(dto.rawContent);
}
// 1. Toggle Share Status
@Patch('/:lessonId/share')
async toggleShareStatus(
  @Param('containerId') containerId: string,
  @Param('lessonId') lessonId: string,
  @Body() dto: UpdateLessonShareDto,
  @NestRequest() req: AuthenticatedRequest,

```

```

    ) {
      return this.lessonService.toggleShareStatus(
        lessonId,
        containerId,
        req.user.sub,
        dto.isPublic,
      );
    }
  }
}

```

Dto:

```

import {
  IsNotEmpty,
  IsString,
  IsArray,
  ValidateNested,
  IsOptional,
} from 'class-validator';
import { Type } from 'class-transformer';
export class CreateTopicDto {
  @IsString()
  @IsNotEmpty({ message: 'Topic title is required' })
  title: string;
  @IsString()
  @IsOptional()
  description?: string | null;
}
export class CreateLessonDto {
  @IsString()
  @IsNotEmpty()
  title: string;
  @IsOptional()
  @IsString()
  @IsNotEmpty()
  rawContent?: string;
  @IsOptional()
  @IsArray()
  @ValidateNested({ each: true })
  @Type(() => CreateTopicDto)
  topics?: CreateTopicDto[];
}

```

```

import {
  IsNotEmpty,
  IsString,
  IsArray,
  ValidateNested,
  IsOptional,
} from 'class-validator';
import { Type } from 'class-transformer';
export class CreateTopicDto {
  @IsString()
  @IsNotEmpty({ message: 'Topic title is required' })
  title: string;
  @IsString()
  @IsOptional()
  description?: string | null;
}
export class CreateLessonDto {
  @IsString()
  @IsNotEmpty()
  title: string;
  @IsOptional()
  @IsString()
  @IsNotEmpty()
  rawContent?: string;
  @IsOptional()
  @IsArray()
  @ValidateNested({ each: true })
  @Type(() => CreateTopicDto)
  topics?: CreateTopicDto[];
}

```

```

import { IsBoolean } from 'class-validator';
export class UpdateLessonShareDto {
  @IsBoolean()
  isPublic: boolean;
}

```

```

import { IsOptional, IsString } from 'class-validator';
export class UpdateLessonDto {
  @IsString()

```

```

@IsOptional()
title?: string;
}

import { IsOptional, IsString, IsNotEmpty } from 'class-validator';
export class UpdateTopicDto {
  @IsOptional()
  @IsString()
  @IsNotEmpty()
  title?: string;
  @IsOptional()
  @IsString()
  description?: string | null;
}

```

Progress module :

```

import {
  Controller,
  Get,
  Put,
  Post,
  Param,
  Body,
  UseGuards,
  Request,
  Patch,
} from '@nestjs/common';
import type { Request as ExpressRequest } from 'express';
import { ProgressService } from '../progress.service.js';
import { UpdateProgressDto } from '../dto/update-progress.dto.js';
import { JwtGuard } from '../auth/guards/jwt.guard.js';
import { UpdateTopicProgressDto } from '../dto/update-topic-progress.dto.js';
interface AuthenticatedRequest extends ExpressRequest {
  user: {
    sub: string;
  };
}
@Controller('containers/:containerId/lessons/:lessonId/progress')
@UseGuards(JwtGuard)
export class ProgressController {
  constructor(private readonly progressService: ProgressService) {}
  /**
   * Get or create progress for a lesson
   * GET /containers/:containerId/lessons/:lessonId/progress
   */
  @Get()
  async getOrCreateProgress(
    @Param('containerId') containerId: string,
    @Param('lessonId') lessonId: string,
    @Request() req: AuthenticatedRequest,
  ) {
    return this.progressService.getOrCreateProgress(
      req.user.sub,
      containerId,
      lessonId,
    );
  }
  /**
   * Update progress ( position, completion status)
   * PUT /containers/:containerId/lessons/:lessonId/progress
   */
  @Put()
  async updateProgress(
    @Param('containerId') containerId: string,
    @Param('lessonId') lessonId: string,
    @Body() updateProgressDto: UpdateProgressDto,
    @Request() req: AuthenticatedRequest,
  ) {
    return this.progressService.updateProgress(
      req.user.sub,
      containerId,
      lessonId,
      updateProgressDto,
    );
  }
  @Patch('topics/:topicId')

```

```

async updateTopicProgress(
  @Param('containerId') containerId: string,
  @Param('lessonId') lessonId: string,
  @Param('topicId') topicId: string,
  @Body() dto: UpdateTopicProgressDto,
  @Request() req: AuthenticatedRequest,
) {
  return this.progressService.updateTopicProgress(
    req.user.sub,
    containerId,
    lessonId,
    topicId,
    dto,
  );
}
/**
 * Get lesson specific progress
 * GET
/containers/:containerId/lessons/:lessonId/progress/details
*/
@Get('details')
async getLessonProgress(
  @Param('containerId') containerId: string,
  @Param('lessonId') lessonId: string,
  @Request() req: AuthenticatedRequest,
) {
  return this.progressService.getLessonProgress(
    req.user.sub,
    containerId,
    lessonId,
  );
}
/**
 * Complete a review session for this lesson
 * POST
/containers/:containerId/lessons/:lessonId/progress/review
*/
@Post('review')
async completeReview(
  @Param('containerId') containerId: string,
  @Param('lessonId') lessonId: string,
  @Request() req: AuthenticatedRequest,
) {
  return this.progressService.completeReview(
    req.user.sub,
    containerId,
    lessonId,
  );
}
}
@Controller('progress')
@UseGuards(JwtGuard)
export class UserProgressController {
  constructor(private readonly progressService:
    ProgressService) {}
  /**
   * Get due reviews for today (Dashboard - Today's Review
   section)
   * GET /progress/due-reviews
   */
  @Get('due-reviews')
  async getDueReviews(@Request() req: AuthenticatedRequest) {
    return this.progressService.getDueReviews(req.user.sub);
  }
  /**
   * Get user's overall statistics
   * GET /progress/stats
   */
  @Get('stats')
  async getUserStats(@Request() req: AuthenticatedRequest) {
    return this.progressService.getUserStats(req.user.sub);
  }
  /**
   * Get all user's progress records
   * GET /progress
   */
  @Get()
  async getUserProgress(@Request() req: AuthenticatedRequest) {
    return this.progressService.getUserProgress(req.user.sub);
  }
}

```

Dto:

```

import { IsOptional, IsNumber, Min } from 'class-validator';
export class UpdateProgressDto {

```

```

@IsNumber()
@IsOptional()
@Min(0)
scrollPosition?: number;
}

import { IsBoolean } from 'class-validator';
export class UpdateTopicProgressDto {
  @IsBoolean()
  isCompleted: boolean;
}

```

Quiz module :

```

import {
  Controller,
  Post,
  Get,
  Body,
  Param,
  UseGuards,
  Req,
} from '@nestjs/common';
import type { Request } from 'express';
import { QuizService } from '../quiz.service.js';
import { CreateQuizDto } from '../dto/create-quiz.dto.js';
import { SubmitQuizDto } from '../dto/submit-quiz.dto.js';
import { JwtGuard } from '../auth/guards/jwt.guard.js';
import { ParseUUIDPipe } from '@nestjs/common';
// 2. الخاصة بالطلب Interface
interface AuthenticatedRequest extends Request {
  user: {
    sub: string;
    [key: string]: any;
  };
};

@Controller('containers/:containerId/lessons/:lessonId/quizzes')
@UseGuards(JwtGuard)
export class QuizController {
  constructor(private readonly quizService: QuizService) {}
  /**
   * Create and generate a new quiz
   * POST /containers/:containerId/lessons/:lessonId/quizzes
   */
  @Post()
  async createQuiz(
    @Param('containerId') containerId: string,
    @Param('lessonId') lessonId: string,
    @Body() createQuizDto: CreateQuizDto,
    @Req() req: AuthenticatedRequest,
  ) {
    return this.quizService.createQuiz(
      req.user.sub,
      containerId,
      lessonId,
      createQuizDto,
    );
  }
  /**
   * Get a quiz with questions (without correct answers)
   * GET /containers/:containerId/lessons/:lessonId/quizzes/:id
   */
  @Get('/:id')
  async getQuiz(
    @Param('containerId', ParseUUIDPipe) containerId: string,
    @Param('lessonId', ParseUUIDPipe) lessonId: string,
    @Param('id', ParseUUIDPipe) quizId: string,
    @Req() req: AuthenticatedRequest,
  ) {
    return this.quizService.getQuiz(
      quizId,
      lessonId,
      containerId,
      req.user.sub,
    );
  }
  /**
   * Submit quiz answers and get results
   * POST
   */
  @Post('/:id/submit')

```

```

async submitQuiz(
  @Param('containerId') containerId: string,
  @Param('lessonId') lessonId: string,
  @Param('id') quizId: string,
  @Body() submitQuizDto: SubmitQuizDto,
  @Req() req: AuthenticatedRequest,
) {
  return this.quizService.submitQuiz(
    quizId,
    req.user.sub,
    containerId,
    lessonId,
    submitQuizDto,
  );
}
}
}

```

Dto:

```

import {
  IsNotEmpty,
  IsEnum,
  IsArray,
  IsOptional,
  IsUUID,
  IsInt,
  Min,
  Max,
} from 'class-validator';
export enum QuizScope {
  FULL_LESSON = 'full_lesson',
  SELECTED_TOPICS = 'selected_topics',
  RANDOM = 'random',
}
export enum QuizDifficulty {
  EASY = 'easy',
  MEDIUM = 'medium',
  HARD = 'hard',
}
export enum QuizFormat {
  MULTIPLE_CHOICE = 'multiple_choice',
  OPEN_TEXT = 'open_text',
  FILL_BLANKS = 'fill_blanks',
  TRUE_FALSE = 'true_false',
}
export class CreateQuizDto {
  @IsEnum(QuizScope)
  @IsNotEmpty({ message: 'Scope is required' })
  scope: QuizScope;
  @IsArray()
  @IsOptional()
  @IsUUID('4', { each: true })
  selectedTopicIds?: string[];
  @IsEnum(QuizDifficulty)
  @IsNotEmpty({ message: 'Difficulty is required' })
  difficulty: QuizDifficulty;
  @IsEnum(QuizFormat)
  @IsNotEmpty({ message: 'Format is required' })
  format: QuizFormat;
  @IsInt()
  @Min(1, { message: 'Question count must be at least 1' })
  @Max(50, { message: 'Question count cannot exceed 50' })
  questionCount: number;
}

```

```

import {
  IsNotEmpty,
  IsArray,
  ValidateNested,
  IsUUID,
  ArrayMinSize,
  IsString
} from 'class-validator';
import { Type } from 'class-transformer';
export class QuizAnswerDto {
  @IsUUID('4')
  @IsNotEmpty({ message: 'Question ID is required' })
  questionId: string;
  @IsString()
  @IsNotEmpty({ message: 'User answer is required' })
  userAnswer: string;
}
export class SubmitQuizDto {
  @IsUUID('4')
  @IsNotEmpty({ message: 'Quiz ID is required' })

```

```

quizId: string;
@isArray()
@arrayMinSize(1)
@validateNested({ each: true })
@type(() => QuizAnswerDto)
answers: QuizAnswerDto[];
}

```

Search module :

```

import {
  Controller,
  Get,
  Query,
  Param,
  UseGuards,
  Request,
} from '@nestjs/common';
import type { Request as ExpressRequest } from 'express';
import { SearchService } from '../search.service.js';
import { SearchLessonDto } from '../dto/search-lesson.dto.js';
import { SearchTopicDto } from '../dto/search-topic.dto.js';
import { JwtGuard } from '../auth/guards/jwt.guard.js';
interface AuthenticatedRequest extends ExpressRequest {
  user: {
    sub: string;
    [key: string]: any;
  };
}
@Controller('containers/:containerId/search')
@UseGuards(JwtGuard)
export class SearchController {
  constructor(private readonly searchService: SearchService) {}
  /**
   * Search lessons in a specific container
   * GET /containers/:containerId/search/lessons
   */
  @Get('lessons')
  async searchLessonsInContainer(
    @Param('containerId') containerId: string,
    @Query() filters: SearchLessonDto,
    @Request() req: AuthenticatedRequest,
  ) {
    return this.searchService.searchLessonsInContainer(
      req.user.sub,
      containerId,
      filters,
    );
  }
  /**
   * Search topics in a specific lesson
   * GET
   /containers/:containerId/search/lessons/:lessonId/topics
   */
  @Get('lessons/:lessonId/topics')
  async searchTopicsInLesson(
    @Param('containerId') containerId: string,
    @Param('lessonId') lessonId: string,
    @Query() filters: SearchTopicDto,
    @Request() req: AuthenticatedRequest,
  ) {
    return this.searchService.searchTopicsInLesson(
      req.user.sub,
      containerId,
      lessonId,
      filters,
    );
  }
  /**
   * Search topics across all lessons in a container
   * GET /containers/:containerId/search/topics
   */
  @Get('topics')
  async searchTopicsAcrossLessons(
    @Param('containerId') containerId: string,
    @Query() filters: SearchTopicDto,
    @Request() req: AuthenticatedRequest,
  ) {
    return this.searchService.searchTopicsAcrossLessons(
      req.user.sub,
      containerId,
      filters,
    );
  }
}

```

```

}
@Controller('search')
@UseGuards(JwtGuard)
export class GlobalSearchController {
  constructor(private readonly searchService: SearchService) {}
  /**
   * Global search across all user lessons
   * GET /search/lessons
   */
  @Get('lessons')
  async searchLessonsGlobal(
    @Query() filters: SearchLessonDto,
    @Request() req: AuthenticatedRequest,
  ) {
    return this.searchService.searchLessonsAcrossContainers(
      req.user.sub,
      filters,
    );
  }
}

```

```

Dto:
import { IsOptional, IsString, IsIn, IsInt, Min } from 'class-validator';
import { Type } from 'class-transformer';
export class SearchLessonDto {
  @IsString()
  @IsOptional()
  query?: string;
  @IsString()
  @IsOptional()
  @IsIn(['in_progress', 'mastered', 'all'])
  status?: 'in_progress' | 'mastered' | 'all';
  @IsString()
  @IsOptional()
  @IsIn(['newest', 'oldest', 'most_completed'])
  sortBy?: 'newest' | 'oldest' | 'most_completed';
  @IsOptional()
  @Type(() => Number)
  @IsInt()
  @Min(0)
  skip?: number = 0;
  @IsOptional()
  @Type(() => Number)
  @IsInt()
  @Min(1)
  take?: number = 10;
}

```

```

import { IsOptional, IsString, IsIn, IsInt, Min } from 'class-validator';
import { Type } from 'class-transformer';
export class SearchTopicDto {
  @IsString()
  @IsOptional()
  query?: string;
  @IsString()
  @IsOptional()
  @IsIn(['completed', 'not_completed', 'all'])
  status?: 'completed' | 'not_completed' | 'all';
  @IsString()
  @IsOptional()
  @IsIn(['newest', 'oldest'])
  sortBy?: 'newest' | 'oldest';
  @IsOptional()
  @Type(() => Number)
  @IsInt()
  @Min(0)
  skip?: number = 0;
  @IsOptional()
  @Type(() => Number)
  @IsInt()
  @Min(1)
  take?: number = 10;
}

```

Summary module :

```

import {
  Controller,
  Post,

```

```

    Get,
    Delete,
    Body,
    Param,
    UseGuards,
    Request,
} from '@nestjs/common';
import { Request as ExpressRequest } from 'express';
import { SummaryService } from '../summary.service.js';
import { CreateSummaryDto } from '../dto/create-summary.dto.js';
import { JwtGuard } from '../auth/guards/jwt.guard.js';
// Define custom request interface if req.user is populated by
JwtGuard
interface AuthenticatedRequest extends ExpressRequest {
  user: {
    sub: string;
    [key: string]: any;
  };
}
@Controller('containers/:containerId/lessons/:lessonId/summaries')
@UseGuards(JwtGuard)
export class SummaryController {
  constructor(private readonly summaryService: SummaryService) {}

  @Post()
  async createSummary(
    @Param('containerId') containerId: string,
    @Param('lessonId') lessonId: string,
    @Body() createSummaryDto: CreateSummaryDto,
    @Request() req: AuthenticatedRequest,
  ) {
    return this.summaryService.createSummary(
      req.user.sub,
      containerId,
      lessonId,
      createSummaryDto,
    );
  }

  @Get()
  async getSummaries(
    @Param('containerId') containerId: string,
    @Param('lessonId') lessonId: string,
    @Request() req: AuthenticatedRequest,
  ) {
    return this.summaryService.getSummaries(
      lessonId,
      containerId,
      req.user.sub,
    );
  }

  @Get('/:id')
  async getSummary(
    @Param('containerId') containerId: string,
    @Param('lessonId') lessonId: string,
    @Param('id') summaryId: string,
    @Request() req: AuthenticatedRequest,
  ) {
    return this.summaryService.getSummary(
      summaryId,
      lessonId,
      containerId,
      req.user.sub,
    );
  }

  @Delete('/:id')
  async deleteSummary(
    @Param('containerId') containerId: string,
    @Param('lessonId') lessonId: string,
    @Param('id') summaryId: string,
    @Request() req: AuthenticatedRequest,
  ) {
    return this.summaryService.deleteSummary(
      summaryId,
      lessonId,
      containerId,
      req.user.sub,
    );
  }
}

```

Dto:

```

import { IsNotEmpty, IsEnum, IsArray, IsOptional, IsUUID } from
'class-validator';
export enum SummaryDepth {

```

```

SHORT = 'short',
MEDIUM = 'medium',
DETAILED = 'detailed',
}
export class CreateSummaryDto {
  @IsEnum(SummaryDepth)
  @NotEmpty({ message: 'Depth is required' })
  depth: SummaryDepth;
  @IsArray()
  @IsOptional()
  @IsUUID('4', { each: true })
  selectedTopicIds?: string[];
}

```

Ask module :

```

import {
  Controller,
  Post,
  Get,
  Delete,
  Body,
  Param,
  UseGuards,
  Request,
} from '@nestjs/common';
import type { Request as ExpressRequest } from 'express';
import { AskService } from '../ask.service.js';
import { CreateQuestionDto } from '../dto/create-question.dto.js';
import { JwtGuard } from '../auth/guards/jwt.guard.js';
interface AuthenticatedRequest extends ExpressRequest {
  user: {
    sub: string;
  };
};
@Controller('containers/:containerId/lessons/:lessonId/ask')
@UseGuards(JwtGuard)
export class AskController {
  constructor(private readonly askService: AskService) {}
  /**
   * Ask a question about a lesson
   * POST /containers/:containerId/lessons/:lessonId/ask
   */
  @Post()
  async askQuestion(
    @Param('containerId') containerId: string,
    @Param('lessonId') lessonId: string,
    @Body() createQuestionDto: CreateQuestionDto,
    @Request() req: AuthenticatedRequest,
  ) {
    return this.askService.askQuestion(
      req.user.sub,
      containerId,
      lessonId,
      createQuestionDto,
    );
  }
  /**
   * Get all questions for a lesson
   * GET /containers/:containerId/lessons/:lessonId/ask
   */
  @Get()
  async getLessonQuestions(
    @Param('containerId') containerId: string,
    @Param('lessonId') lessonId: string,
    @Request() req: AuthenticatedRequest,
  ) {
    return this.askService.getLessonQuestions(
      lessonId,
      containerId,
      req.user.sub,
    );
  }
  /**
   * Get a specific question
   * GET /containers/:containerId/lessons/:lessonId/ask/:id
   */
  @Get('/:id')
  async getQuestion(

```

```

    @Param('containerId') containerId: string,
    @Param('lessonId') lessonId: string,
    @Param('id') questionId: string,
    @Request() req: AuthenticatedRequest,
  ) {
    return this.askService.getQuestion(
      questionId,
      lessonId,
      containerId,
      req.user.sub,
    );
  }
  /**
   * Delete a question
   * DELETE /containers/:containerId/lessons/:lessonId/ask/:id
   */
  @Delete('/:id')
  async deleteQuestion(
    @Param('containerId') containerId: string,
    @Param('lessonId') lessonId: string,
    @Param('id') questionId: string,
    @Request() req: AuthenticatedRequest,
  ) {
    return this.askService.deleteQuestion(
      questionId,
      lessonId,
      containerId,
      req.user.sub,
    );
  }
}
@Controller('questions')
@UseGuards(JwtGuard)
export class UserQuestionsController {
  constructor(private readonly askService: AskService) {}
  /**
   * Get all user's questions across all lessons
   * GET /questions
   */
  @Get()
  async getUserQuestions(@Request() req: AuthenticatedRequest)
  {
    return this.askService.getUserQuestions(req.user.sub);
  }
}

```

Dto:

```

import { IsNotEmpty, IsString, IsOptional, IsUUID } from
'class-validator';
export class CreateQuestionDto {
  @IsString()
  @IsNotEmpty({ message: 'Question text is required' })
  questionText: string;
  @IsUUID()
  @IsOptional()
  topicId?: string;
}

```

Annotation module:

```

import {
  Controller,
  Get,
  Post,
  Patch,
  Delete,
  Param,
  Body,
  UseGuards,
  Request,
} from '@nestjs/common';
import { AnnotationService } from './annotation.service.js';
import { CreateAnnotationDto } from './dto/create-
annotation.dto.js';
import { UpdateAnnotationDto } from './dto/update-
annotation.dto.js';
import { JwtGuard } from '../auth/guards/jwt.guard.js';
import type { Request as ExpressRequest } from 'express';
interface AuthenticatedRequest extends ExpressRequest {
  user: {
    sub: string;
  };
}

```

```
//ask summary search quiz analytics gemini
@Controller('lessons/:lessonId/annotations')
@UseGuards(JwtGuard)
export class AnnotationController {
  constructor(private readonly annotationService:
AnnotationService) {}
  @Post()
  async create(
    @Param('lessonId') lessonId: string,
    @Request() req: AuthenticatedRequest,
    @Body() dto: CreateAnnotationDto,
  ) {
    return await this.annotationService.create(req.user.sub,
lessonId, dto);
  }
  @Get()
  async findByLesson(
    @Param('lessonId') lessonId: string,
    @Request() req: AuthenticatedRequest,
  ) {
    return await
this.annotationService.findByLesson(req.user.sub, lessonId);
  }
  @Patch(':id')
  async update(
    @Param('id') id: string,
    @Request() req: AuthenticatedRequest,
    @Body() dto: UpdateAnnotationDto,
  ) {
    return await this.annotationService.update(id,
req.user.sub, dto);
  }
  @Delete(':id')
  async remove(@Param('id') id: string, @Request() req:
AuthenticatedRequest) {
    return await this.annotationService.remove(id,
req.user.sub);
  }
}
```

Dto:

```
import {
  IsNotEmpty,
  IsString,
  IsOptional,
  IsUUID,
} from 'class-validator';
export class CreateAnnotationDto {
  @IsString()
  @IsNotEmpty({ message: 'Content is required' })
  content: string;
  @IsOptional()
  @IsUUID()
  topicId?: string;
  @IsOptional()
  @IsString()
  color?: string;
}
```

```
import { IsOptional, IsString } from 'class-validator';
export class UpdateAnnotationDto {
  @IsOptional()
  @IsString()
  content?: string;
  @IsOptional()
  @IsString()
  color?: string;
}
```

Analytics module :

Only controller :

```
import { Controller, Get, UseGuards, Request } from
'@nestjs/common';
import { AnalyticsService } from './analytics.service.js';
import { JwtGuard } from '../auth/guards/jwt.guard.js';
import type { Request as ExpressRequest } from 'express';
interface AuthenticatedRequest extends ExpressRequest {
  user: {
    sub: string;
  };
}
@Controller('analytics')
@UseGuards(JwtGuard)
```

```

export class AnalyticsController {
  constructor(private readonly analyticsService:
AnalyticsService) {}
  @Get('overview')
  async getOverview(@Request() req: AuthenticatedRequest) {
    return
this.analyticsService.getUserDashboardOverview(req.user.sub);
  }
}

```

tayeb ana 3nde fekra tenye (tech stack : react with type script and):

StudyLoop	dark/lightmode	name and login or logout
Home Library Reviews Quizzes summary Analytics		MAIN CONTENT
Profile name and emoji cart (jemdin mesh buton)		

bel home : Dashboard
هذه أهم صفحة.
dashboard 20 مليانة card.
أول شيء:
Good evening, Hussein.
Continue learning where you left off.
ثم:

Continue Learning	
Software Architecture	
Clean Architecture	
72%	
Continue →	

ثم:
Quick stats
12 Lessons 84 Topics 67% Completion
ويعددين:
Review Due
Review Due
3 lessons are ready for review.
Clean Architecture Review →
SOLID Review →
Dependency Injection Review →

w bel library:
4. Library
knowledge workspace تكون مثل.

My Library	
+ New	
Search...	
Software Eng.	Backend
12 lessons	8 lessons
68% progress	42% progress

تكون هادئة جداً.Container cards.لكن أنا أفضل ال
Container:ولما تدخل:

Software Engineering
12 Lessons

Search lessons...

Clean Architecture

8 / 10 topics completed

80%

SOLID

5 / 5 topics completed

100%

5. Lesson Page — أهم شاشة
هون لازم يكون التصميم ممتاز
شيء 500 فيها masidebar بدني
شي مثل:

← Software Engineering

Clean Architecture
8 / 10 topics completed - 80%

- 01 What is Clean Architecture?
text...
- 02 Dependency Rule
text...
- 03 Boundaries
text...

w kmn : إنشاء 8. Lesson
أساسية. feature يكون ممتاز لأنه UX هون لازم
تقليدي طويل: form بدل

Create Lesson
How do you want to add your lesson?

♦ Structured | ♦ Paste Content

Add topics | Let AI structure

manually | your notes

إذا اختر: Structured:

Lesson title
Topic 1
[title]
[description]
+ Add topic
Create Lesson
إذا اختر: Paste:

Paste your notes

Paste your study material here...

♦ Structure with AI
بالمشروع. هذه يمكن تكون من أجمل أجزاء ال
w bade fiha kmn fo2 bel sheshe : generate summary : bas yekbesa byes2alo 2eza bado yeha short ,
medium aw detailed w byes2alo 2eza bado yeha 3an kel l topic aw baso yna2e shi topic , 2eza bado
yna2e shi topic mnefta7lo l page la 7ata yna2e l topic li bado yeha
w kmn generate quiz : 2eza byekbesa mnes2alo shu l scope li bado yeh wel difficulty wel quiz format
w fi kmn go to quiz page , bas yekbesa , men7awlo 3ala l quizzes page w mnfawto 3ala l quiz li n3amalo
generate aw byo3dulo l quiz men l ajdad lal a2dam aw 7asba l lesson krmel yfut d8re 3a quiz li 3emela
generate w bi 7ela w by3mela submit
w kmn fi button fo2 2esmo go to summary : bas yekbesa bi raw7u 3al summary page w byo3rdulo l
summary men l ajdad lal a2dam aw 7asab l lesson krmel yfut d8re 3al summary li 3emela generate
w fi kmn ask krmel bas yekbesa ye5du 3al ask w yes2al li bado yeh d8re men dun ma yotla3 yerja3 yfut
w hek mnkun Salasna men l libraries
2amma bel nesbe lal review : 10. Reviews
واضحة جداً. Feature هذه لازم تكون
صفحة:

Reviews
You have 4 lessons to review today.

Clean Architecture

Due today

8 / 10 topics completed

Start Review →

وعند الدخول:

Review: Clean Architecture
What is the Dependency Rule?
[Think about your answer]
Show explanation
I remember ✓
Need more review
بصير واضح للمستخدم. StudyLoop وهنا فعلياً ال

w bel nesbe lal summary wel quiz fa kmn bi kuno 7asab l lesson aw men l ajdad lal 2a2dam mtl ma bdk

2ama bel nesbe lal analytics:

bade page ktir ktir 7lwe w fiha ktir rasmw w 2osas 7elwe lal analytics w hayda l service Zeza bi hemak:

```
import { Injectable, Inject } from '@nestjs/common';
import { LESSON_REPOSITORY } from '../lesson/repositories/lesson.repository.interface.js';
import type { ILessonRepository } from '../lesson/repositories/lesson.repository.interface.js';
import { TOPIC_REPOSITORY } from '../lesson/repositories/topic.repository.interface.js';
import type { ITopicRepository } from '../lesson/repositories/topic.repository.interface.js';
import { ANNOTATION_REPOSITORY } from '../annotation/repositories/annotation.repository.interface.js';
import type { IAnnotationRepository } from '../annotation/repositories/annotation.repository.interface.js';
import { QUIZ_REPOSITORY } from '../quiz/repositories/quiz.repository.interface.js';
import type { IQuizRepository } from '../quiz/repositories/quiz.repository.interface.js';
import { PROGRESS_REPOSITORY } from '../progress/repositories/progress.repository.interface.js';
import type { IProgressRepository } from '../progress/repositories/progress.repository.interface.js';
@Injectable()
export class AnalyticsService {
  constructor(
    @Inject(LESSON_REPOSITORY)
    private readonly lessonRepository: ILessonRepository,
    @Inject(TOPIC_REPOSITORY)
    private readonly topicRepository: ITopicRepository,
    @Inject(ANNOTATION_REPOSITORY)
    private readonly annotationRepository: IAnnotationRepository,
    @Inject(QUIZ_REPOSITORY)
    private readonly quizRepository: IQuizRepository,
    @Inject(PROGRESS_REPOSITORY)
    private readonly progressRepository: IProgressRepository,
  ) {}
  async getUserDashboardOverview(userId: string) {
    // 1. Total Lessons & Global Topics Progress
    const totalLessons = await this.lessonRepository.countByUserId(userId);
    const totalTopics = await this.topicRepository.countByUserId(userId);
    const completedTopics =
      await this.topicRepository.countCompletedByUserId(userId);
    const globalCompletionRate =
      totalTopics > 0 ? Math.round((completedTopics / totalTopics) * 100) : 0;
    // 2. Recent 5 Lessons Progress (Zedna l-Array la ZeSer 5 drous)
    const recentLessons = await this.lessonRepository.findRecentByUserId(
      userId,
      5,
    );
    const recentActivity = await Promise.all(
      recentLessons.map(async (lesson) => {
        const progress = await this.progressRepository.findByUserAndLesson(
          userId,
          lesson.id,
        );
      })
    );
    return {
      lessonId: lesson.id,
      lessonTitle: lesson.title,
      containerId: lesson.containerId,
      totalTopics: progress?.totalTopicsCount ?? lesson.topics.length,
      completedTopics: progress?.completedTopicsCount ?? 0,
      progressPercentage: progress?.completionPercentage ?? 0,
      lastAccessedAt: lesson.updatedAt,
    };
  },
  // 3. Quiz Performance Stats
  const quizStats = await this.quizRepository.getUserQuizStats(userId);
  // 4. Annotations Count
  const totalAnnotations =
    await this.annotationRepository.countByUserId(userId);
  return {
    overview: {
      totalLessons,
      totalTopics,
      completedTopics,
      globalCompletionRate,
      totalAnnotations,
    },
    recentActivity,
    quizzes: {
      totalTaken: quizStats?.totalTaken ?? 0,
      averageScore: quizStats?.averageScore ?? 0,
    },
  };
}
```



desktop design ok!!!!